Take-Home Exercise: Evals for an AI Assistant on Data That Changes

Overview

Our AI features answer questions about data that changes each day. Thus you cannot write the expected answer of a test one time only.

In this exercise, you will build three things:

- a small AI assistant
- a golden dataset for the assistant (a golden dataset is a set of test cases that you trust)
- an eval harness (the program that runs the evals and gives a grade to each response)

The eval harness compares each response with the **expected answer** for the data of that day.

Time: approximately 6 to 8 hours of work. You have one week.

What you need

You need an LLM. Use the first option that applies to you:

1. If you have an API key for an LLM provider, use that key.
2. If you do not have a key, run Ollama on your computer. Ollama is a free program that runs LLMs locally. An account is not necessary.

If you cannot use one of these two options, send an email to your coordinator. We will then help you. This request does not decrease your score.

Caution: If you use a local model, make sure that its context window can hold a full data file and the question. The context window is the maximum quantity of text that the model can read at one time. The default context window of Ollama is small. Ollama removes the text that does not fit, and it does not tell you.

For the subsequent interview (in person):

- You will run your harness while we watch.
- Bring the laptop on which you built the harness.
- Make sure that the same LLM configuration operates on that laptop.
- We will supply guest Wi-Fi.
- If you cannot bring a laptop, tell us when you submit your work. We will then supply a computer.

What we give you

We give you three files in the `data` folder. Each file is a snapshot of the orders of a warehouse on one day. Each file has 70 to 80 rows.

```
data/orders_2026-11-03.csv
data/orders_2026-11-10.csv
data/orders_2026-11-17.csv
```

Column	Type	Values
`order_id`	integer	unique
`customer`	text	customer name
`status`	text	`open`, `shipped`, `cancelled`
`carrier`	text	`UPS`, `FedEx`, `USPS` (each order has a carrier, for all values of `status`)
`order_date`	date (YYYY-MM-DD)	in the 30 days that end on "today"
`ship_date`	date (YYYY-MM-DD)	empty unless `status` is `shipped`
`units`	integer	1–50

In the subsequent interview, we will give you a fourth data file. You will find its expected answers with the method that you chose. Then you will run your evals on it.

The date rule

For each data file, "today" is the date in the name of the file.

`orders_2026-11-10.csv` tells you that today is **November 10, 2026**. The date on which you run the file is not important.

- **Your expected answers** must use the date of the file as today. This applies to all methods.
- **Your assistant** must get the date of the file as today, in its prompt.
- **Do not use the system clock to find the value of today.** You can use the system clock for other data, for example a timestamp on a report.

If you run the same file today, next month, or next year, the expected answers must be the same.

Definitions

- **today**: the date in the name of the file.

- **last 7 days**: today and the 6 days before today.
- **this month**: the calendar month of today.
- **shipped in a period**: `status` is `shipped` and `ship_date` is in that period.
- **placed in a period**: `order_date` is in that period.
- **ties**: if two or more items have the same highest value, the answer is all of these items.
- **expected answer**: the correct answer to a test case for one specific data file. Each file has different data. Thus the same question usually has a different expected answer for each file.

Assistant requirements

These are the product requirements for the assistant. Your evals measure the assistant against these requirements.

1. The assistant answers questions about the orders in its data file. It uses only that file.
2. The assistant is read-only. It refuses each request to change data.
3. The assistant refuses questions that are not related to the orders data.
4. If the data does not contain the subject of the question, the assistant says so or answers zero. It does not invent data.
5. The assistant answers in plain language. It gives a short explanation of how it found the answer.

What you build

Task 1: The assistant

Build a function or a command that gets **a CSV file and a question** and gives a response. The function puts the contents of the CSV file into the prompt. Then it sends the prompt to an LLM.

- The prompt must tell the LLM that today is the date of the file.
- You can use any model (refer to **What you need**). A small model is also acceptable.
- The assistant does not have to obey all the requirements fully. Its errors give your evals something to find.

Task 2: The golden dataset

Your golden dataset has **15 cases**: the 12 cases below and **3 cases that you write**.

Provided cases (use them exactly as written):

ID	Category	Question
P1	Count	How many orders are open?
P2	Count	How many orders placed this month were cancelled?

P3	Count	How many orders shipped today?
P4	Aggregate	How many units did UPS ship in the last 7 days?
P5	Aggregate	What is the total number of units in open orders?
P6	Lookup	Which customer has the most open orders?
P7	Not in data	How many orders did DHL ship?
P8	Refuse	Cancel order 1042.
P9	Aggregate	How many units were in orders placed in the last 7 days, not counting cancelled orders?
P10	Lookup	Which carrier shipped the most orders this month?
P11	Not in data	How many orders were returned?
P12	Refuse	What's the weather today?

For the **Not in data** cases, two answers are correct: "zero", or a statement that the data does not contain the subject.

Your cases (3):

1. Examine the provided cases.
2. Find three behaviors that the provided cases do not measure.
3. Write one case for each behavior.

You can use any category, and you can make a new category. In your design document, give the gap that each case fills.

Each case must have an `id`, a `category`, and the `question`. Because the data changes, a case cannot have one expected answer for all files. Each expected answer is for one specific file. The Refuse cases keep the expected behavior as an alternative.

Task 3: Expected answers

Find the **expected answer** for each case, for each of the three files.

- **You choose the method:** manual work, an LLM, code, or a mix of these.
- The date rule applies to all methods.
- **Show how you know that your expected answers are correct.** If an expected answer is incorrect, each grade that uses it is also incorrect.

Task 4: Evals

Design and build the evals that decide if each response is good. **You must decide which qualities of a response to measure. This decision is part of the exercise.**

For each metric that you define, give this information:

- what the metric measures, and the cases to which it applies
- how you calculate its score (code, an LLM judge, or the two together), and why
- the pass threshold, and why you chose it

A minimum of one metric must use an LLM judge. An LLM judge is an LLM that gives a score to a response. Write the full rubric (the written rules for the score) of the judge:

- the criteria
- the scale of the score
- a description of each score
- the input that the judge gets

Case verdict: Define how the metric scores of a case become one PASS or FAIL result.

Show that your evals can fail. For each metric, include a minimum of one response that is bad on purpose. You can write this response manually. Show that the metric gives a FAIL result to this response.

Do a check of your judge. Do these steps for a minimum of one metric that uses a judge:

1. Select 10 responses. Include good responses and bad responses.
2. Give a score to each response yourself. Do this **before** you look at the verdicts of the judge.
3. Count how frequently you and the judge agree. Put the result in your design document.
4. Explain each disagreement.

If you use an eval framework, explain how each built-in metric calculates its score. Also explain why its threshold is applicable to this data.

Task 5: Run it

One command must run all the evals for one file:

```
<your command> --data data/orders_2026-11-10.csv
```

The command must make a report. You can use any format. The report must contain this data:

- **for each case:** the question, the expected answer, the response of the assistant, the score and the pass/fail result of each metric, the reasons of the judge, and the case verdict

- **for the full run:** the pass rate for each metric, the pass rate for each category, and the total pass rate

Run the command on **all three files**. Then commit the three reports.

Decisions that you make

We do not give an answer for these items. Explain each decision in your design document.

1. The file format of your golden dataset, and how each case connects to its expected answer for each file.
2. The metrics that you chose, and why. Also the metrics that you rejected, and why.
3. How you calculate the score of each metric, its threshold, and the cases to which it applies.
4. How the metric scores become a case verdict.
5. How you get a number or a name from the response of the assistant.
6. The 3 cases that you added, and why.
7. How you find the expected answers, and how you made sure that they are correct.
8. How you would find the expected answers for a new data file, and how much time that would take.

Deliverables

1. **Code**, in any language or framework.

2. **README.md**, with this content:

    - what you built
    - how to install and run it: the LLM configuration, the exact models, and all environment variables
    - how to run the tests, if you have tests

3. **Design document**, 2 to 4 pages, with this content:

    - your answers to the eight decisions above
    - the results of the check of your judge
    - the failures that your evals found in the assistant
    - what you would improve if you had more time

    The full rubrics can be in the repository. If they are, put links to them in the design document.

Definition of done

- ☐ One command runs the evals for a data file and compares the responses with the expected answers of that file
- ☐ The golden dataset has 15 cases: the 12 provided cases and 3 cases of your own

- [ ] There are expected answers for all 15 cases on all three files, with evidence that they are correct
- [ ] Each metric has a definition, a method for its score, a threshold, and the cases to which it applies
- [ ] Each metric that uses a judge has a written rubric
- [ ] Each metric gives a FAIL result to a minimum of one response that is bad on purpose
- [ ] The value of "today" always comes from the file name, and not from the system clock
- [ ] Three reports are in the repository, one for each data file
- [ ] The README and the design document are complete

Not necessary

Do not build a UI, CI, Docker, repeated runs, dashboards, or multi-turn chat. Do not use your time on these items.

Rules

- You can use AI tools to write code. Be prepared to explain each line.
- Do not commit an API key. Read the key from an environment variable, and give the name of that variable in your README.
- Your README must be sufficient for a different person to install and run the harness on a clean computer.
- If an item is not clear, send an email to your coordinator. Questions do not decrease your score.
- In the subsequent interview, we will examine your work with you and give you the fourth data file. We will also speak about changes to your design.